

Password Strength Analyzer

Internship Mini-Project Report — Cybersecurity / Python Track

Project Title	Password Strength Analyzer
Domain	Cybersecurity & Applied Cryptography
Language / Stack	Python 3 (re, hashlib, secrets, sqlite3)
Deliverables	password_strength_analyzer.py, this report, README

1. Objective

The goal of this project is to build a tool that evaluates how strong a user-supplied password is, explains its weaknesses in plain language, and suggests a stronger alternative. As an optional extension, the tool integrates with a small local database so that a user cannot reuse a password they have used before — a common requirement in real-world account security policies. Building this tool is a practical way to learn the fundamentals of password security and basic cryptographic concepts such as hashing, salting, and entropy.

2. Background Concepts

2.1 Why password strength matters

Weak passwords are one of the leading causes of account compromise. Attackers use techniques such as **dictionary attacks** (trying common words and leaked passwords), **brute-force attacks** (trying every possible combination), and **credential stuffing** (reusing passwords leaked from other breaches). A good strength analyzer helps a user avoid all three by catching common passwords, enforcing complexity, and preventing reuse.

2.2 Entropy — measuring randomness

Password strength is often measured in bits of **entropy**, which estimates how many guesses an attacker would need. It is approximated as:

```
Entropy (bits) = Length x log2(Pool Size)

where Pool Size is the number of possible
characters (e.g. 26 lowercase + 26 uppercase
+ 10 digits + 32 symbols = 94)
```

A password with 60+ bits of entropy is generally considered strong against offline brute-force attempts; 80+ bits is very strong.

2.3 Hashing and salting

The tool never stores plaintext passwords. Instead, when checking for reuse it stores a **SHA-256 hash** of the password combined with a random, unique **salt** per entry. Salting prevents attackers from using precomputed "rainbow tables" to reverse the hash, and ensures that two identical passwords produce different stored hashes. (For production authentication systems, a slow, memory-hard algorithm such as **bcrypt**, **scrypt**, or **Argon2** should be used instead of a fast hash like SHA-256 — this project uses SHA-256 for simplicity since it is only comparing password history, not storing login credentials.)

3. Key Features Implemented

- **Length check** — flags passwords under 8 characters, rewards length up to ~14+ characters.
- **Complexity check** — verifies presence of lowercase, uppercase, digits, and special characters.

- **Uniqueness check** — flags common/breached-style passwords, keyboard sequences (e.g. 'qwerty'), repeated characters (e.g. 'aaa'), and passwords containing the username.
- **Entropy estimation** — computes an approximate bits-of-entropy score.
- **Composite scoring** — combines the above into a 0–100 score with a label (Very Weak / Weak / Moderate / Strong / Very Strong).
- **Stronger password suggestion** — generates a cryptographically secure random password using Python's secrets module, plus a memorable Diceware-style passphrase alternative.
- **Optional reuse prevention** — stores a salted SHA-256 hash of each accepted password in a local SQLite database, and blocks future passwords that match a past one.

4. Scoring Methodology

The total score (max 100) is the sum of three sub-scores:

Category	Max Points	What it checks
Length	30	Character count vs. minimum (8) and ideal (14) length
Complexity	40	10 pts each for lowercase, uppercase, digits, symbols present
Uniqueness	30	Deductions for common passwords, sequences, repeats, reused history

Score Range	Label
85–100	Very Strong
65–84	Strong
45–64	Moderate
25–44	Weak
0–24	Very Weak

5. Tool Architecture

The tool is implemented as a single Python module with a clear separation of concerns:

- **PasswordStrengthAnalyzer** — the main class holding all scoring logic.
- **PasswordReport** — a dataclass capturing scores, issues, and suggestions for one password.
- **SQLite layer** (`password_history.db`) — stores salted hashes per username for reuse detection; created automatically on first run.
- **CLI demo** (`main()`) — an interactive command-line loop for testing the analyzer without building a UI.

5.1 Core snippet — complexity scoring

```
def _score_complexity(self, password, report):
    has_lower = bool(re.search(r"[a-z]", password))
    has_upper = bool(re.search(r"[A-Z]", password))
    has_digit = bool(re.search(r"\d", password))
    has_symbol = bool(re.search(r"[^\w\s]", password))
    categories_used = sum([has_lower, has_upper, has_digit, has_symbol])
    report.complexity_score = categories_used * 10 # up to 40
```

5.2 Core snippet — salted hashing for reuse prevention

```
def _hash_password(self, password, salt):
    return hashlib.sha256((salt + password).encode()).hexdigest()

def is_password_reused(self, username, password):
    rows = ... # fetch (salt, hash) pairs for this user
    return any(
        self._hash_password(password, salt) == stored_hash
        for salt, stored_hash in rows
    )
```

6. Sample Run / Output

Below is example output from running the analyzer on four representative passwords:

Password	Score	Label	Entropy
password	35 / 100	Weak	37.6 bits
Passw0rd!	91 / 100	Very Strong*	59.0 bits
correcthorsebatterystaple	70 / 100	Strong	117.5 bits
Tr0ub4dor&3xample!Zz	100 / 100	Very Strong	131.1 bits

*Short (9 chars) but hits all 4 complexity categories, illustrating why length alone doesn't determine strength — pairing length with complexity gives the best score.

Console output for the weak password "password":

```
=====
Strength Score : 35/100 -> Weak
Estimated Entropy : 37.6 bits
-----
Length score : 20/30
Complexity score : 10/40
Uniqueness score : 5/30
-----
Issues found:
- Missing uppercase letters.
- Missing digits.
- Missing special characters (e.g. ! @ # $ %).
- This is one of the most common passwords in the world.
-----
Suggestions:
- Use at least 12-14 characters.
- Mix uppercase, lowercase, digits, and symbols.
- Avoid dictionary words, names, and predictable sequences.
- Never reuse a password across sites or over time.

Example strong alternative : qzQywEx0RE&Q5ivv
Example passphrase : Coral-Juniper-Coral-Cinder95
=====
```

7. Key Learnings

- How to quantify password strength using length, character-class complexity, and entropy rather than relying on a single rule of thumb.
- Why plaintext passwords should never be stored, and how salted hashing (SHA-256 here, bcrypt/Argon2 in production) protects stored credentials.
- The difference between a fast hash (good for checksums, unsuitable for password storage) and a slow, memory-hard hash (good for password storage).

- How simple regex-based heuristics can catch common attack patterns (dictionary words, keyboard walks, repeated characters).
- How to design a small local database (SQLite) to enforce a password-history / no-reuse policy, a standard control in enterprise password policies (e.g. NIST SP 800-63B).

8. Possible Future Improvements

- Replace the local common-password list with a live check against the 'Have I Been Pwned' Pwned Passwords API (k-anonymity model, so the full password is never sent).
- Swap SHA-256 for bcrypt/Argon2 if the database were ever used for real authentication, not just reuse-checking.
- Add a simple web UI (Flask/Streamlit) with a live strength meter as the user types.
- Support configurable organizational password policies (e.g. minimum length, required categories).

9. How to Run

```
git clone <your-repo-url>
cd password-strength-analyzer
pip install -r requirements.txt # no external deps beyond stdlib
python password_strength_analyzer.py
```

The script launches an interactive CLI: enter a username once, then enter passwords to analyze repeatedly. Type 'quit' to exit. Passwords scoring 45+ can optionally be saved (as a salted hash) to demonstrate the reuse-prevention feature on a second run.

End of report — see `password_strength_analyzer.py` for the full source code.